Filière :	Développement des Systèmes d'Information	Durée :	2 Heures
Épreuve :	POO/CS	Coefficient :	

ÉTUDE DE CAS : GESTION DE SCOLARTIÉ

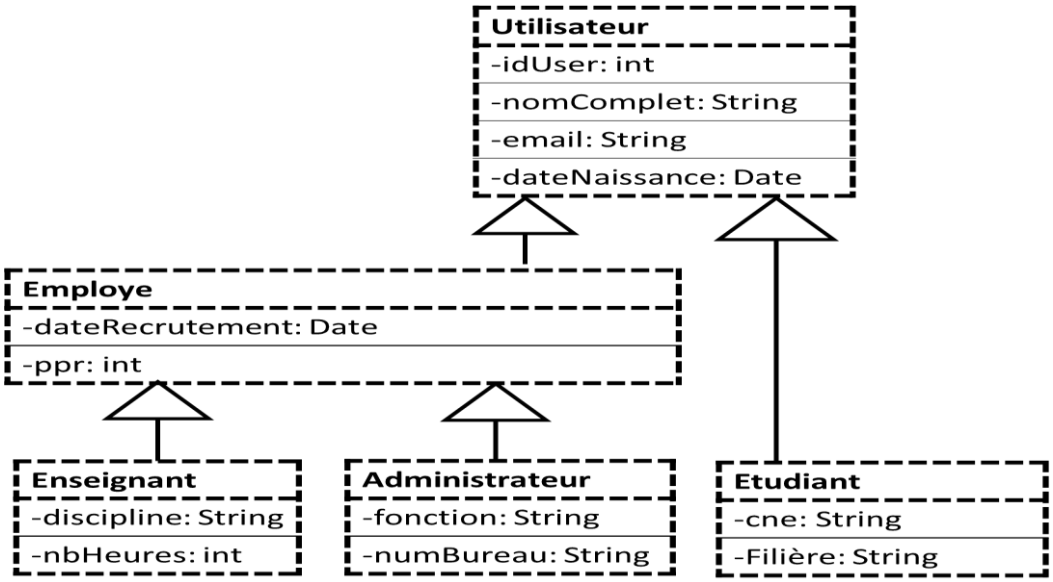
Le centre de préparation du Brevet Technicien Supérieur Alkendi dispose d'un Système d'information puissant qui facilite la communication entre les différents acteurs pédagogiques, à savoir : les étudiants, les enseignants et les administrateurs.

Le service IT du centre propose des solutions informatiques développées en fonction des besoins collectés à partir de ces différents acteurs ; parmi ces solutions, on trouve :

- **La partie 1 :** Une application desktop pour la gestion des utilisateurs.
- **La partie 2 :** Une application Distribuée qui permet de gérer les étudiants dans une base de données MySQL, avec des fonctionnalités CRUD et des requêtes spécifiques.

Partie 1 : PROGRAMMATION ORIENTEE OBJET (JAVA) *(20 pts):*

Le diagramme de classe ci-dessous projette de la lumière sur les principales entités de l'application.



La classe **Utilisateur** contient les caractéristiques d'un utilisateur. La figure ci-dessous montre le squelette de cette classe.

```
public class Utilisateur implements Serializable, Comparable<Utilisateur> {
   private int idUser;
   private String nomComplet, email ;
   private Date dateNaissance;
   private static int nbUsers = 0;

      public Utilisateur(String nomComplet, String email,Date dateNaissance) { ……. }
      public static int countUsers() {……. }
      public static int incrUser() {…….}
      public String verifEmail(String email) {…….}
      @Override
      public String toString() {…….}}
      @Override
      public int compareTo(Utilisateur autre) {….}
```

Donner le code de :

1- La méthode **incrIdUser** qui permet d'incrémenter la variable **nbUsers** et de retourner ce dernier. **(0,5pt)**
2- La méthode **verifEmail** qui permet de vérifier si un email passé en paramètre est correct.
   - ✓ Si l'email contient la chaine **'@bts-kendi.ma'**, on retourne l'email.
   - ✓ Si l'email ne contient pas la chaine **'@bts-kendi.ma'**, on affiche le message suivant **"Attention email incorrect : email non ajouté"** et on retourne **null.** **(1,5pt)**
3- Le Constructeur de la classe **Utilisateur** avec 3 paramètres qui vérifie : **(1,5pt)**
   - ✓ Si l'email passé en paramètre est correct, il initialise les trois attributs (nomComplet, email et dateNaissance) par les valeurs passées en paramètres.
   - ✓ Sinon, il initialise seulement les 2 attributs (nomComplet et dateNaissance) et affiche le message **"Attention email incorrect : email non ajouté"**.

   🔔 **Remarque :** l'id de l'utilisateur est auto-incrémenté. Utilisez pour cela la fonction **incrIdUser**

4- La méthode statique **countUsers** qui permet de retourner le nombre d'objets utilisateurs instanciés. **(0,5pt)**
5- La méthode **toString** qui permet de retourner le nom et l'email de l'utilisateur **(0,5pt)**
6- Les getters et les setters des attributs idUser et NomComplet. **(1,5pt)**
7- La méthode **compareTo** pour permettre le tri par nom. **(1pt)**

La classe **Employe** contient les caractéristiques spécifiques d'un employé. La figure ci-dessous montre le squelette de cette classe. :

```
public class Employe extends Utilisateur implements Serializable {
   private Date dateRecrutement;
   private int ppr;

   public Employe(Date dateRecrutement, int ppr, String nomComplet, String email, Date
dateNaissance) {…… }

   public boolean equals(Employe e) {…… }

   @Override
   public int getIdUser(){  }

   @Override
    public String toString() { …… }
}
```

Donner le code de :

8- Le constructeur avec 5 paramètres **(1 pt)**
9- La méthode **equals** qui permet de comparer l'objet courant avec un autre objet en fonction de son idUser : elle retourne **true** si les deux objets ont le même idUser et retourne false dans le cas contraire. **(1pt)**
10- La méthode **getIdUser** qui permet de retourner l'idUser de l'employé. **(0,5pt)**

11- La méthode **toString** qui permet de retourner tous les attributs d'un employé (utiliser la méthode toString de la classe mère) **(1 pt)**

La classe **Enseignant** contient les caractéristiques spécifiques d'un Enseignant. La figure ci-dessous montre le squelette de cette classe.

```
public class Enseignant extends Employe implements Serializable {
   private String discipline;
   private int nbHeure;

   public Enseignant(String discipline, int nbHeure, Date dateRecrutement, int ppr, String
nomComplet, String email, Date dateNaissance) {……. }
   public void isOfTheSameClassAs(Object o) {…….}
   public void afficher() {…… }
    @Override
    public String toString() {…… }
}
```

Donner le code de :

1- Le constructeur avec 7 paramètres. **(1pt)**
2- La méthode **isOfTheSameClassAs** qui permet de déterminer si un objet passé en paramètre est de même type que l'objet courant, dans ce cas elle renvoie **true** sinon elle renvoie **false.** **(1pt)**
3- La méthode **toString** qui permet de retourner tous les attributs d'un Enseignant (utiliser la méthode toString de la classe mère). **(1pt)**
4- La méthode **Afficher** qui permet d'afficher toutes les cordonnés d'un enseignant **(1pt)**

La classe **GestionUtilisateurs** gère la collection d'utilisateurs, le tri, et la lecture/écriture dans un fichier. La figure ci-dessous montre le squelette de cette classe. :

```
public class GestionUtilisateurs {
        private List<Utilisateur> utilisateurs;
   public GestionUtilisateurs () {…… }
   public void ajouterUtilisateur(Utilisateur utilisateur) {…… }
   public void trierUtilisateurs () {…… }
   public void afficherUtilisateurs () {…… }
   public void sauvegarderDansFichier(String nomFichier) {…… }
   public chargerDepuisFichier(String nomFichier) {…… }
}
```

Donner le code de :

12- Le constructeur sans paramètres **(0,5pt)**
13- La méthode **ajouterUtilisateur** pour ajouter un utilisateur à la collection. **(1pt)**
14- La méthode **trierUtilisateurs** pour trier la collection par nom. **(1pt)**
15- La méthode **afficherUtilisateurs** pour afficher tous les utilisateurs. **(1 pt)**
16- Les méthodes **sauvegarderDansFichier** et **chargerDepuisFichier** permettent de sauvegarder et charger les utilisateurs depuis un fichier. **(2 pts)**

Partie 2 : Développement d'application Client/serveur (JAVA) (9 pts)

La base de données de l'établissement se trouve dans un serveur MySQL ayant l'adresse IP **192.168.1.33**. L'utilisateur ayant le login **'adminDB'** et le mot de passe **'admin123@Com'** a tous les privilèges nécessaires pour manipuler la base de données nommée **scolarite**, dont vous trouverez un extrait dans le tableau ci-dessous.

Table : etudiant :

Colonne	Type	Clé Primaire
cne	varchar(10)	Oui
Tel	varchar(15)	Non
email	varchar(20)	Non
MoyGenerale	float	Non

1- Donner le code de la méthode statique **connect** qui permet de se connecter à la base de données. Elle retourne la variable de connexion en cas de succès et null dans le cas contraire. **(1pt)**

```
public static Connection connect() {.....}
```

2- Donner le code de la méthode statique **getEmailByCne** qui permet de retourner l'email et l'étudiant en fonction du cne et de la variable de connexion passés en paramètre. **(1pt)**

```
public static String getEmailByCne(String cne, Connection con) throws SQLException {.....}
```

3- Donner le code de la méthode statique **getTelByCne** qui permet de retourner le numéro de téléphone de l'étudiant en fonction du cne et de la variable de connexion passés en paramètre. **(1pt)**

```
public static String getTelByCne(String cne, Connection con) throws SQLException {.....}
```

4- Donner le code de la méthode statique **getResult** qui permet de retourner la moyenne générale d'un étudiant dont le cne et la variable de connexion sont passés en paramètre. **(1pt)**

```
public static String getResult(String cne, Connection con) throws SQLException {.....}
```

5- Écrire une méthode updateEmail(String cne, String newEmail, Connection con) qui met à jour l'adresse email d'un étudiant en fonction de son cne. Elle doit retourner true si la mise à jour est réussie, sinon false. **(1 pt)**

6- Écrire une méthode insertStudent(String cne, String tel, String email, float moyGenerale, Connection con) qui insère un nouvel étudiant dans la table etudiant. Elle doit retourner true si l'insertion est réussie, sinon false. **(1 pt)**

7- Écrire une méthode deleteStudent(String cne, Connection con) qui supprime un étudiant de la table etudiant en fonction de son cne. Elle doit retourner true si la suppression est réussie, sinon false. **(1 pt)**

8- Écrire une méthode getAllStudents(Connection con) qui récupère et affiche tous les étudiants présents dans la table. **(1 pt)**

9- Écrire une méthode studentExists(String cne, Connection con) qui vérifie si un étudiant existe dans la base de données. Elle doit retourner true si l'étudiant existe, sinon false. **(1 pt)**

10- Ajouter une gestion robuste des exceptions pour chaque méthode JDBC. **(1 pt)**